# SQL Project : Retail Store

## Level 1: Basics

1. **Retrieve customer names and emails for email marketing**

   SELECT name, email FROM customers;

| name | email |
|---|---|
| Thomas Owens | user1@example.com |
| Charles Grant | user2@example.com |
| Kaitlin Richards | user3@example.com |
| Christina Williams | user4@example.com |
| David Allen | user5@example.com |
| Mark Duke | user6@example.com |
| Briana Wright | user7@example.com |
| John Bryan | user8@example.com |
| Jason Thompson | user9@example.com |

2. **View complete product catalog with all available details**

   SELECT * FROM products;

| product_id | name | category | price | stock_quantity | added_on |
|---|---|---|---|---|---|
| 1 | Plant No | Home | 639.43 | 152 | 2024-01-30 06:30:53 |
| 2 | Population Social | Clothing | 4813.68 | 84 | 2025-05-30 10:02:50 |
| 3 | Available Answer | Electronics | 2529.51 | 101 | 2025-04-13 01:11:46 |
| 4 | Any Question | Clothing | 4759.28 | 179 | 2025-06-03 13:34:03 |
| 5 | Natural Network | Toys | 4722.66 | 75 | 2023-11-06 00:47:37 |
| 6 | If Whatever | Electronics | 177.40 | 64 | 2024-12-19 10:37:14 |
| 7 | Response Indeed | Clothing | 4897.36 | 36 | 2025-03-29 02:43:08 |
| 8 | Every Amount | Home | 4173.60 | 156 | 2025-04-30 03:11:10 |
| 9 | Common Study | Toys | 985.19 | 171 | 2023-07-20 13:06:42 |

3. **List all unique product categories**

   SELECT DISTINCT category FROM products;

| category |
|---|
| Home |
| Clothing |
| Electronics |
| Toys |
| Books |

# SQL Project : Retail Store

4. **Show all products priced above ₹1,000**

   SELECT name FROM products WHERE price > 1000;

| name |
|---|
| Population Social |
| Available Answer |
| Any Question |
| Natural Network |
| Response Indeed |
| Every Amount |
| Development System |
| Build Her |
| Action Ask |

5. **Display products within a mid-range price bracket (₹2,000 to ₹5,000)**

   SELECT name FROM products WHERE price >=2000 and price <=5000;

| name |
|---|
| Population Social |
| Available Answer |
| Any Question |
| Natural Network |
| Response Indeed |
| Every Amount |
| Development System |
| Action Ask |
| Full West |

6. **Fetch data for specific customer IDs (e.g., from loyalty program list)**

   SELECT * FROM customers WHERE customer_id IN (1, 2, 3, 4, 5, 6);

# SQL Project : Retail Store

| customer_id | name | email | phone | created_at |
|---|---|---|---|---|
| 1 | Thomas Owens | user1@example.com | 142-479-1945 | 2024-10-14 16:01:12 |
| 2 | Charles Grant | user2@example.com | 9153947511 | 2023-11-25 15:45:24 |
| 3 | Kaitlin Richards | user3@example.com | 2073473421 | 2024-06-23 09:55:22 |
| 4 | Christina Williams | user4@example.com | 586-605-5061x06 | 2024-10-27 17:19:38 |
| 5 | David Allen | user5@example.com | (751)456-8289x1 | 2023-10-29 02:43:00 |
| 6 | Mark Duke | user6@example.com | (144)957-2811 | 2024-06-24 03:22:59 |
| NULL | NULL | NULL | NULL | NULL |

7. **Identify customers whose names start with the letter 'A'**
   SELECT * FROM customers WHERE name LIKE "A%";

| customer_id | name | email | phone | created_at |
|---|---|---|---|---|
| 15 | Austin Flores | user15@example.com | 329.901.1576x66 | 2024-06-13 09:03:42 |
| 16 | Amy Landry | user16@example.com | +1-278-019-3748 | 2024-02-28 17:51:50 |
| 19 | Amanda Bright | user19@example.com | 380.981.9798x69 | 2024-12-20 22:58:15 |
| 27 | Adrienne Green | user27@example.com | 530.644.8455x93 | 2023-08-22 01:55:29 |
| NULL | NULL | NULL | NULL | NULL |

8. **List electronics products priced under ₹3,000**
   SELECT name FROM products WHERE category = "Electronics" and price <=3000;

| name |
|---|
| Available Answer |
| If Whatever |
| Place Low |
| Series Page |
| Despite Win |
| Actually Term |
| Southern Thing |

9. **Display product names and prices in descending order of price**
   SELECT name, price FROM products ORDER BY price desc;

| name | price |
|---|---|
| Response Indeed | 4897.36 |
| Population Social | 4813.68 |
| Development System | 4801.78 |
| Any Question | 4759.28 |
| Fire Often | 4734.89 |
| Natural Network | 4722.66 |
| Build High | 4707.14 |
| Serious Recognize | 4523.10 |
| Study Total | 4413.68 |

10. **Display product names and prices, sorted by price and then by name**

    SELECT name, price FROM products ORDER BY price, name;

| name | price |
|---|---|
| If Whatever | 177.40 |
| Listen Development | 296.17 |
| Actually Term | 396.11 |
| Television Stock | 421.73 |
| Southern Thing | 512.46 |
| Plant No | 639.43 |
| Place Low | 723.97 |
| Stock Article | 834.75 |
| Toward Minute | 857.85 |

## Level 2: Filtering and Formatting

1. **Retrieve orders where customer information is missing (possibly due to data migration or deletion)**

   SELECT * FROM customers WHERE customer_id IS NULL;

| customer_id | name | email | phone | created_at |
|---|---|---|---|---|
| NULL | NULL | NULL | NULL | NULL |

# SQL Project : Retail Store

2. **Display customer names and emails using column aliases for frontend readability**

   SELECT name AS customer_name, email AS customer_email FROM customers;

| customer_name | customer_email |
|---|---|
| Thomas Owens | user1@example.com |
| Charles Grant | user2@example.com |
| Kaitlin Richards | user3@example.com |
| Christina Williams | user4@example.com |

3. **Calculate total value per item ordered by multyplying quantity and item price**

   SELECT *, quantity*item_price AS total_value FROM order_items;

| order_item_id | order_id | product_id | quantity | item_price | total_value |
|---|---|---|---|---|---|
| 1 | 1 | 19 | 2 | 4707.14 | 9414.28 |
| 2 | 2 | 6 | 3 | 177.40 | 532.20 |
| 3 | 3 | 9 | 3 | 985.19 | 2955.57 |
| 4 | 3 | 23 | 1 | 2208.99 | 2208.99 |
| 5 | 4 | 35 | 2 | 4734.89 | 9469.78 |
| 6 | 5 | 19 | 1 | 4707.14 | 4707.14 |

4. **Combine customer name and phone number in a single column**

   SELECT CONCAT(name, "-", phone) AS customer_directory FROM customers;

| customer_directory |
|---|
| Thomas Owens-142-479-1945 |
| Charles Grant-9153947511 |
| Kaitlin Richards-2073473421 |
| Christina Williams-586-605-5061x06 |
| David Allen-(751)456-8289x1 |
| Mark Duke-(144)957-2811 |
| Briana Wright-223-833-9635 |

5. **Extract only the date part from order timestamps for date-wise reporting**

   SELECT *, DATE(order_date) AS date FROM orders;

| | order_id | customer_id | order_date | status | total_amount | date |
|---|---|---|---|---|---|---|
| ▶ | 1 | 20 | 2025-03-02 07:20:11 | Delivered | 9414.28 | 2025-03-02 |
| | 2 | 18 | 2024-10-09 18:08:21 | Shipped | 532.20 | 2024-10-09 |
| | 3 | 15 | 2025-05-08 00:08:27 | Cancelled | 5164.56 | 2025-05-08 |
| | 4 | 11 | 2024-09-19 22:16:13 | Delivered | 9469.78 | 2024-09-19 |
| | 5 | 12 | 2025-04-08 18:02:06 | Pending | 14501.86 | 2025-04-08 |
| | 6 | 29 | 2024-10-25 07:33:59 | Cancelled | 31050.17 | 2024-10-25 |

6. **List products that do not have any stock left**

   SELECT * FROM products WHERE stock_quantity = 0;

| | product_id | name | category | price | stock_quantity | added_on |
|---|---|---|---|---|---|---|
| * | NULL | NULL | NULL | NULL | NULL | NULL |

## Level 3: Aggregations

1. **Count the total number of orders placed**

   SELECT COUNT(order_id) AS order_count FROM orders;

| | order_count |
|---|---|
| ▶ | 400 |

2. **Calculate the total revenue collected from all orders**

   SELECT SUM(total_amount) AS total_revenue FROM orders;

| | total_revenue |
|---|---|
| ▶ | 6960973.66 |

3. **Calculate the average order value**

   SELECT AVG(total_amount) AS avg_order_value FROM orders;

| | avg_order_value |
|---|---|
| ▶ | 17402.434150 |

4. **Count the number of customers who have placed at least one order**
   SELECT COUNT(DISTINCT customer_id) FROM orders;

| | COUNT(DISTINCT customer_id) |
|---|---|
| ▶ | 30 |

5. **Find the number of orders placed by each customer**
   SELECT customer_id, COUNT(order_id) FROM orders GROUP BY customer_id;

| | customer_id | COUNT(order_id) |
|---|---|---|
| ▶ | 1 | 12 |
| | 2 | 17 |
| | 3 | 17 |
| | 4 | 9 |

6. **Find total sales amount made by each customer**
   SELECT customer_id, SUM(total_amount) FROM orders GROUP BY customer_id;

| | customer_id | SUM(total_amount) |
|---|---|---|
| ▶ | 1 | 183747.44 |
| | 2 | 284420.07 |
| | 3 | 253783.31 |
| | 4 | 137562.22 |
| | 5 | 262504.19 |

7. **List the number of products sold per category**
   SELECT category, COUNT(product_id) FROM products GROUP BY category;

| category | COUNT(product_id) |
|---|---|
| Home | 8 |
| Clothing | 13 |
| Electronics | 14 |
| Toys | 8 |
| Books | 7 |

8. **Find the average item price per category**

   SELECT category, AVG(price) FROM products GROUP BY category;

| category | AVG(price) |
|---|---|
| Home | 2146.367500 |
| Clothing | 3434.581538 |
| Electronics | 2653.388571 |
| Toys | 2516.526250 |
| Books | 3167.084286 |

9. **Show number of orders placed per day**

   SELECT DATE(order_date), COUNT(order_id) FROM orders GROUP BY DATE(order_date);

| DATE(order_date) | COUNT(order_id) |
|---|---|
| 2025-03-02 | 3 |
| 2024-10-09 | 2 |
| 2025-05-08 | 2 |
| 2024-09-19 | 3 |
| 2025-04-08 | 3 |

10. **List total payments received per payment method**

    SELECT method, SUM(amount_paid) FROM payments GROUP BY method;

| method | SUM(amount_paid) |
|---|---|
| Credit Card | 1754603.10 |
| Net Banking | 1658383.90 |
| UPI | 1617408.78 |
| Debit Card | 1930577.88 |

## Level 4: Multi-Table Queries (JOINS)

1. **Retrieve order details along with the customer name (INNER JOIN)**

```
SELECT c.name, o.*
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id;
```

| name | order_id | customer_id | order_date | status | total_amount |
|---|---|---|---|---|---|
| Thomas Owens | 14 | 1 | 2024-09-24 21:21:38 | Shipped | 15803.34 |
| Thomas Owens | 17 | 1 | 2024-08-19 21:17:57 | Pending | 11173.77 |
| Thomas Owens | 61 | 1 | 2024-12-26 23:21:58 | Shipped | 13053.00 |
| Thomas Owens | 76 | 1 | 2025-01-14 22:59:54 | Pending | 23506.81 |
| Thomas Owens | 92 | 1 | 2024-09-26 11:33:58 | Shipped | 834.75 |
| Thomas Owens | 109 | 1 | 2025-03-17 04:56:18 | Shipped | 12190.14 |

2. **Get list of products that have been sold (INNER JOIN with order_items)**

```
SELECT distinct p.name
FROM products p
JOIN order_items oi
ON p.product_id = oi.product_id;
```

| name |
|---|
| Plant No |
| Population Social |
| Available Answer |
| Any Question |
| Natural Network |
| If Whatever |

# SQL Project : Retail Store

3. **List all orders with their payment method (INNER JOIN)**

```
SELECT o.order_id, p.method
FROM orders o
JOIN payments p
ON o.order_id = p.order_id;
```

| order_id | method |
|---|---|
| 14 | UPI |
| 17 | UPI |
| 61 | Net Banking |
| 76 | Credit Card |
| 92 | Net Banking |
| 109 | UPI |

4. **Get list of customers and their orders (LEFT JOIN)**

```
SELECT c.name, o.*
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

| name | order_id | customer_id | order_date | status | total_amount |
|---|---|---|---|---|---|
| Thomas Owens | 14 | 1 | 2024-09-24 21:21:38 | Shipped | 15803.34 |
| Thomas Owens | 17 | 1 | 2024-08-19 21:17:57 | Pending | 11173.77 |
| Thomas Owens | 61 | 1 | 2024-12-26 23:21:58 | Shipped | 13053.00 |
| Thomas Owens | 76 | 1 | 2025-01-14 22:59:54 | Pending | 23506.81 |
| Thomas Owens | 92 | 1 | 2024-09-26 11:33:58 | Shipped | 834.75 |

5. **List all products along with order item quantity (LEFT JOIN)**

```
SELECT p.name, oi.quantity
FROM products p
LEFT JOIN order_items oi
ON p.product_id = oi.product_id;
```

| name | quantity |
|---|---|
| Plant No | 1 |
| Plant No | 3 |
| Plant No | 1 |
| Plant No | 1 |
| Plant No | 2 |

6. **List all payments including those with no matching orders (RIGHT JOIN)**

```
SELECT p.*, o.order_id
FROM orders o
RIGHT JOIN payments p
ON o.order_id = p.order_id;
```

| payment_id | order_id | payment_date | amount_paid | method | order_id |
|---|---|---|---|---|---|
| 1 | 1 | 2025-03-02 08:16:11 | 9414.28 | Credit Card | 1 |
| 2 | 2 | 2024-10-09 19:00:21 | 532.20 | Net Banking | 2 |
| 3 | 3 | 2025-05-08 00:55:27 | 5164.56 | Credit Card | 3 |
| 4 | 4 | 2024-09-19 22:28:13 | 9469.78 | UPI | 4 |
| 5 | 5 | 2025-04-08 18:26:06 | 14501.86 | UPI | 5 |

7. **Combine data from three tables: customer, order, and payment**

```
SELECT c.*, o.*, p.*
FROM customers c
RIGHT JOIN orders o
ON c.customer_id = o.customer_id
JOIN payments p
ON o.order_id = p.order_id;
```

| customer_id | name | email | phone | created_at | order_id | customer_id | order_date | status | total_amount | payment_id | order_id | payment_date | amount_paid | metho |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 20 | Megan Lee | user20@example.com | 7044478333 | 2024-07-09 16:13:14 | 1 | 20 | 2025-03-02 07:20:11 | Delivered | 9414.28 | 1 | 1 | 2025-03-02 08:16:11 | 9414.28 | Credit |
| 18 | Jeffrey Bray | user18@example.com | 164.962.0222x92 | 2024-07-02 19:51:55 | 2 | 18 | 2024-10-09 18:08:21 | Shipped | 532.20 | 2 | 2 | 2024-10-09 19:00:21 | 532.20 | Net Ba |
| 15 | Austin Flores | user15@example.com | 329.901.1576x66 | 2024-06-13 09:03:42 | 3 | 15 | 2025-05-08 00:08:27 | Cancelled | 5164.56 | 3 | 3 | 2025-05-08 00:55:27 | 5164.56 | Credit |
| 11 | Walter Jenkins | user11@example.com | 536-329-0817x71 | 2023-10-26 03:12:30 | 4 | 11 | 2024-09-19 22:16:13 | Delivered | 9469.78 | 4 | 4 | 2024-09-19 22:28:13 | 9469.78 | UPI |
| 12 | Mary Knight | user12@example.com | 361-636-3802 | 2023-08-16 20:05:50 | 5 | 12 | 2025-04-08 18:02:06 | Pending | 14501.86 | 5 | 5 | 2025-04-08 18:26:06 | 14501.86 | UPI |
| 29 | Kara Zavala | user29@example.com | 434-985-5776x61 | 2023-09-27 15:57:09 | 6 | 29 | 2024-10-25 07:33:59 | Cancelled | 31050.17 | 6 | 6 | 2024-10-25 08:16:59 | 31050.17 | UPI |

# SQL Project : Retail Store

## Level 5: Subqueries (Inner Queries)

1. **List all products priced above the average product price**

   SELECT * FROM products WHERE price >
   (SELECT AVG(price) FROM products);

| product_id | name | category | price | stock_quantity | added_on |
|---|---|---|---|---|---|
| 2 | Population Social | Clothing | 4813.68 | 84 | 2025-05-30 10:02:50 |
| 4 | Any Question | Clothing | 4759.28 | 179 | 2025-06-03 13:34:03 |
| 5 | Natural Network | Toys | 4722.66 | 75 | 2023-11-06 00:47:37 |
| 7 | Response Indeed | Clothing | 4897.36 | 36 | 2025-03-29 02:43:08 |
| 8 | Every Amount | Home | 4173.60 | 156 | 2025-04-30 03:11:10 |

2. **Find customers who have placed at least one order**

   SELECT customer_id, name FROM customers
   WHERE customer_id IN (SELECT customer_id FROM orders);

| customer_id | name |
|---|---|
| 1 | Thomas Owens |
| 2 | Charles Grant |
| 3 | Kaitlin Richards |
| 4 | Christina Williams |
| 5 | David Allen |

3. **Show orders whose total amount is above the average for that customer**

   SELECT * FROM
   (SELECT *, AVG(total_amount) OVER(PARTITION BY customer_id) AS avg_amount FROM orders) t
   WHERE total_amount > avg_amount;

| order_id | customer_id | order_date | status | total_amount | avg_amount |
|---|---|---|---|---|---|
| 14 | 1 | 2024-09-24 21:21:38 | Shipped | 15803.34 | 15312.286667 |
| 76 | 1 | 2025-01-14 22:59:54 | Pending | 23506.81 | 15312.286667 |
| 127 | 1 | 2025-06-10 18:01:32 | Pending | 20160.64 | 15312.286667 |
| 278 | 1 | 2025-02-01 20:05:38 | Delivered | 32015.16 | 15312.286667 |
| 379 | 1 | 2025-03-22 15:57:58 | Cancelled | 21586.89 | 15312.286667 |
| 223 | 2 | 2024-11-25 01:32:18 | Cancelled | 41020.75 | 16730.592353 |
| 230 | 2 | 2024-07-25 22:19:17 | Pending | 22655.32 | 16730.592353 |

# SQL Project : Retail Store

4. **Display customers who haven't placed any orders**

   SELECT name FROM customers WHERE customer_id NOT IN
   (SELECT customer_id FROM orders);

| | name |
|---|---|

5. **Show products that were never ordered**

   SELECT name FROM products WHERE product_id NOT IN
   (SELECT product_id FROM order_items);

| | name |
|---|---|

6. **Show highest value order per customer**

   SELECT *
   FROM (SELECT customer_id, max(total_amount) AS max_amount FROM
   orders GROUP BY customer_id) t;

| | customer_id | max_amount |
|---|---|---|
| ▶ | 1 | 32015.16 |
| | 2 | 42056.04 |
| | 3 | 41679.11 |
| | 4 | 25747.34 |
| | 5 | 39921.78 |

7. **Highest Order Per Customer (Including Names)**

   SELECT c.customer_id, c.name, t.max_amount
   FROM customers c
   JOIN (SELECT customer_id, MAX(total_amount) AS max_amount FROM
   orders GROUP BY customer_id) t
   ON c.customer_id = t.customer_id;

| customer_id | name | max_amount |
|---|---|---|
| 1 | Thomas Owens | 32015.16 |
| 2 | Charles Grant | 42056.04 |
| 3 | Kaitlin Richards | 41679.11 |

Level 6: Set Operations

1. **List all customers who have either placed an order or written a product review**

   SELECT customer_id FROM orders
   UNION
   SELECT customer_id FROM product_reviews;

| customer_id |
|---|
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |

2. **List all customers who have placed an order as well as reviewed a product**

   SELECT DISTINCT customer_id FROM orders
   WHERE customer_id IN
   (SELECT customer_id FROM product_reviews);

| customer_id |
|---|
| 1 |
| 2 |
| 4 |
| 6 |
| 7 |
| 9 |
| 10 |

# SQL Project : Retail Store

Schema :

```
create schema mini_project;

use mini_project;


CREATE TABLE customers (

customer_id INT PRIMARY KEY AUTO_INCREMENT,

name VARCHAR(100) NOT NULL,

email VARCHAR(100) NOT NULL UNIQUE,

phone VARCHAR(15),

created_at DATETIME DEFAULT CURRENT_TIMESTAMP

);


CREATE TABLE products (

product_id INT PRIMARY KEY AUTO_INCREMENT,

name VARCHAR(100) NOT NULL,

category VARCHAR(50) NOT NULL,

price DECIMAL(10,2) NOT NULL,

stock_quantity INT NOT NULL DEFAULT 0,

added_on DATETIME DEFAULT CURRENT_TIMESTAMP

);


CREATE TABLE orders (

order_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
customer_id INT,
order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
status VARCHAR(20) DEFAULT 'Pending',
total_amount DECIMAL(10,2),
FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

CREATE TABLE order_items (
order_item_id INT PRIMARY KEY AUTO_INCREMENT,
order_id INT,
product_id INT,
quantity INT NOT NULL CHECK (quantity > 0),
item_price DECIMAL(10,2) NOT NULL,
FOREIGN KEY (order_id) REFERENCES orders(order_id),
FOREIGN KEY (product_id) REFERENCES products(product_id)
);

CREATE TABLE payments (
payment_id INT PRIMARY KEY AUTO_INCREMENT,
order_id INT,
payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
amount_paid DECIMAL(10,2) NOT NULL CHECK (amount_paid > 0),
method VARCHAR(20) NOT NULL,
```

```
FOREIGN KEY (order_id) REFERENCES orders(order_id)
);

CREATE TABLE product_reviews (
review_id INT PRIMARY KEY AUTO_INCREMENT,
product_id INT,
customer_id INT,
rating INT NOT NULL,
review_text TEXT,
review_date DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (product_id) REFERENCES products(product_id),
FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```